Assignment 13. 2

BELLAMKONDA BHANU PRASAD

BATCH NO : 38

2303A52021

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

CODE:

```python
def calculate_area_and_perimeter(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        tuple: A tuple containing the area and perimeter.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# First rectangle
length = 8
width = 4
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)

# Second rectangle
length = 12
width = 6
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

OUTPUT:

```
length = 12
width = 6
area, perimeter = calculate_area_and_perimeter(length, width)

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER

PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Here we only one function it calculates the area and perimeter . when ever we want area and perimeter just we need to pass two arguments so the function auto matically calculates it.

Task Description -2 (Refactoring – Modularizing Inline

Calculations)

• Task: Use AI to refactor a Python script by extracting repeated

inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

Expected Output:

A modular Python program that uses a reusable function to compute

final amounts for different inputs.

CODE:

```
AIAC_LAB 12.3 .py > ...
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculate the final amount after applying a discount.

    Args:
        amount (float): The original amount.
        discount_rate (float): The discount rate as a decimal (default 0.10 for 10%).

    Returns:
        float: The final amount after discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount

# First calculation
amount = 1000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)

# Second calculation
amount = 2000
final_amount = calculate_final_amount(amount)
print("Final Amount:", final_amount)
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
Final Amount: 900.0
Final Amount: 1800.0
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

The code calcutes the price after the discount.the function need two prices by using those prices the final price is calculated.

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of

Python code that relies on nested loops and repeated conditional

checks.

Instructions:

• Use AI to recommend more efficient data structures or logic

simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True
```

if exists:

print(name, "found")

else:

print(name, "not found")

Expected Output:

An optimized Python program that performs faster lookups using

efficient logic, along with execution performance comparison.

CODE:

AIAC_LAB 12.3 .py > ...

```python
import time

# Original inefficient code
def original_check(students, check_list):
    for name in check_list:
        exists = False
        for student in students:
            if name == student:
                exists = True
        if exists:
            print(name, "found")
        else:
            print(name, "not found")

# Optimized code using set for O(1) lookups
def optimized_check(students, check_list):
    students_set = set(students)
    for name in check_list:
        if name in students_set:
            print(name, "found")
        else:
            print(name, "not found")

students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

print("Original code:")
start = time.time()
original_check(students, check_list)
end = time.time()
print(f"Time taken: {end - start:.6f} seconds")

print("\nOptimized code:")
start = time.time()
optimized_check(students, check_list)
end = time.time()
print(f"Time taken: {end - start:.6f} seconds")
```

OUTPUT:

```
29    original_check(students, check_list)
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 6

PS C:\AI Assissted Coding Assessments>

                                    > & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
Original code:
Charan found
Esha not found
Bhavya found
Time taken: 0.000208 seconds

Optimized code:
Charan found
Esha not found
Bhavya found
Time taken: 0.000164 seconds
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

The code compares two ways to check if names are in a student list.The original uses nested loops, which is slow for large lists.The optimized uses a set for fast lookups and shows better performance.

Task Description -4 (Refactoring – Replacing Hardcoded Values

with Constants)

• Task: Utilize AI assistance to improve maintainability by

replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for

clearer and maintainable calculations..

CODE:

```
AIAC_LAB 12.3 .py > ...
# Define constants for better maintainability
INTEREST_RATE = 5  # in percent
TIME = 2  # in years
DENOMINATOR = 100  # for percentage calculation

# First calculation
principal = 1000
simple_interest = (principal * INTEREST_RATE * TIME) / DENOMINATOR
print("Simple Interest:", simple_interest)

# Second calculation
principal = 2000
simple_interest = (principal * INTEREST_RATE * TIME) / DENOMINATOR
print("Simple Interest:", simple_interest)
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
Simple Interest: 100.0
Simple Interest: 200.0
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Calculates simple interest for principals of 1000 and 2000 using these constants. This replaces hardcoded numbers for better maintainability and readability.

Task Description -5 (Re factoring – Enhancing Variable Naming

and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming

unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful

identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments

that improve readability without altering functionality.

CODE:

```
AIAC_LAB 12.3 .py > ...
# Define the first number
first_number = 50

# Define the second number
second_number = 30

# Calculate the sum of the two numbers
sum_result = first_number + second_number

# Print the result
print(sum_result)
```

OUTPUT:

```
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 12.3 .py"
80
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Here we have two variables. We are adding the two numbers.using the return function oonly.